

5.1 Introduction to Python – Long Answer

Python is a high-level, general-purpose, and powerful programming language widely used in various domains such as IoT, AI, Machine Learning, web development, automation, data science, and networking. Python is known for its **simplicity, readability, versatility, and extensive library support**, which makes it extremely popular among beginners as well as professionals.

1. History and Background of Python

- Python was created by **Guido van Rossum** at **Centrum Wiskunde & Informatica (CWI)** in the Netherlands.
- The first version **Python 1.0** was released in **1991**.
- Python 2.0 was released in **2000**.
- Python 3.0 was introduced in **2008**.
- Python is now maintained by the **Python Software Foundation (PSF)**.

Python was designed with the goal of creating a language that is easy to read, easy to write, and powerful enough to support complex applications.

2. Features of Python

Python has many characteristics that make it one of the most widely used languages.

(1) Simple and Easy to Learn

Python syntax is clear, short, and readable like English.

(2) High-Level Language

The programmer does not need to worry about memory management and hardware-level details.

(3) Interpreted Language

Python code is executed line-by-line, making debugging simpler.

(4) Portable

Python works on:

- Windows

- Linux
- macOS
- Raspberry Pi
- Android (via frameworks)

(5) Object-Oriented

Python supports:

- Classes
- Objects
- Inheritance
- Polymorphism

(6) Large Standard Library

Python has built-in modules like:

- math
- os
- random
- datetime

(7) Extensible and Embeddable

Python can be used with C/C++, Java, and other languages.

(8) Supports Multiple Paradigms

- Functional programming
- Object-oriented programming
- Procedural programming

(9) Free and Open Source

Python is open source and free to download and use.

3. Why Python is Popular in IoT?

Python is considered the **best language for IoT** due to the following reasons:

1. Simple syntax

Easy to write programs for beginners and students.

2. Runs on Raspberry Pi

Python is the default language of Raspberry Pi, the most widely used IoT microcomputer.

3. Supports Sensors and Modules

Python has libraries like:

- RPi.GPIO
- gpiozero
- smbus
- serial
- Adafruit_DHT

These help connect sensors such as:

- DHT11 (temperature)
- PIR (motion)
- Ultrasonic (distance)
- Gas sensors
- Light sensors

4. Supports Networking

Python supports:

- HTTP
- MQTT
- WebSockets
- IoT cloud APIs

5. AI + IoT

Python is the leading language for AI and ML, making it ideal for building intelligent IoT systems.

4. Python Applications (Where Python is Used?)

Python is used in almost all technology fields:

1. IoT (Internet of Things)

- Sensor programming
- Automation
- Smart home projects

2. Web Development

- Django
- Flask

3. Data Science & Machine Learning

- NumPy
- pandas
- scikit-learn
- TensorFlow

4. Automation / Scripting

- System tasks automation
- Testing automation
- File management scripts

5. Cybersecurity

- Ethical hacking tools
- Malware analysis

6. Game Development

- Pygame

7. Mobile and GUI apps

- Kivy
- Tkinter

5. Python Installation on Raspberry Pi

Python comes pre-installed on Raspberry Pi OS:

- **Python 2.x**
- **Python 3.x** (default)

You can check the version using:

```
python3 --version
```

Install packages:

```
sudo apt-get install python3-pip
```

6. Running Python Programs

1. Using Python Interpreter (Interactive mode)

```
python3
>>> print("Hello, Pi")
```

2. Running Python Script

Create a file:

```
nano program.py
```

Run it:

```
python3 program.py
```

7. Advantages of Python

- Easy to learn
 - Fast development
 - Strong library support
 - Cross-platform
 - Ideal for IoT and ML
 - Large community support
-

8. Disadvantages of Python

- Slower than C/C++
 - Not perfect for mobile app development
 - Uses more memory compared to low-level languages
 - Not ideal for real-time applications
-

Conclusion

Python is a powerful, user-friendly, and versatile programming language that plays a major role in modern technologies, especially in IoT. Its simplicity, extensive libraries, and compatibility with Raspberry Pi make it the backbone of IoT development. From automation to artificial intelligence, Python continues to be a preferred choice for developers worldwide.

5.2 Applications of Python – Long Answer

Python is one of the most widely used programming languages in the world because of its simplicity, flexibility, and rich ecosystem of libraries. It supports multiple programming paradigms such as object-oriented, procedural, and functional programming, making it suitable for almost every technological domain.

Python plays an important role in **IoT, AI, ML, Data Science, Web Development, Automation**, and many more industries. It is the most preferred language for engineers, students, researchers, and software developers.

Below are the major applications of Python explained in detail.

1. Web Development

Python is widely used for backend web development due to easy syntax and powerful frameworks.

Popular Python web frameworks

- **Django** (High-level, secure, full-stack framework)
- **Flask** (Lightweight micro-framework)
- **FastAPI** (Fast and modern API development)

Features for web development

- Database connectivity
- Form handling
- Authentication & security
- Server-side logic
- REST API development

Python powers websites like:

- Instagram
 - YouTube
 - Spotify
 - Dropbox
-

2. Data Science and Data Analysis

Python is the **#1 language for data science**.

Important libraries:

- **NumPy** – numerical computing
- **pandas** – data manipulation
- **Matplotlib** – data visualization
- **Seaborn** – advanced charts
- **SciPy** – scientific computing

Applications in data science

- Data cleaning
 - Big data processing
 - Visualization and analytics
 - Statistical modeling
-

3. Machine Learning & Artificial Intelligence

Python dominates the AI/ML industry because of its strong libraries and ease of implementation.

AI/ML libraries:

- **TensorFlow**
- **Keras**
- **PyTorch**
- **scikit-learn**

Applications:

- Image classification
- Speech recognition
- Natural language processing
- Recommendation systems
- Predictive analytics

Python is widely used in ML because:

- It has simple syntax
 - Supports large datasets
 - Has GPU libraries (for deep learning)
-

4. IoT (Internet of Things)

Python is the **most important language for IoT**, especially on **Raspberry Pi**.

Why Python is used in IoT?

- Easy to connect sensors
- Supports GPIO programming
- Lightweight and fast
- Access to network protocols (MQTT, HTTP, TCP)

Libraries for IoT:

- **RPi.GPIO**
- **gpiozero**
- **Adafruit_DHT**
- **serial**
- **smbus**

Python is used in:

- Home automation
 - Smart agriculture
 - Smart parking
 - Security systems
 - Robotics
-

5. Automation and Scripting

Python is excellent for automating repetitive tasks.

Automation examples:

- File and folder automation
- Email automation
- Website scraping
- Logging and monitoring systems
- Testing automation

Python scripts are used in:

- DevOps
- System administration
- Network automation

6. Game Development

Python is used to develop simple and educational games.

Library:

- **Pygame** – used for 2D game development

Examples:

- Puzzle games
- Arcade games
- Learning games for kids

7. Software Development

Python is used for:

- Prototyping
- GUI applications
- System tools

GUI frameworks:

- Tkinter
- PyQt
- Kivy

Python applications include:

- Desktop apps
- Productivity tools
- Small-scale software utilities

8. Cybersecurity and Ethical Hacking

Python is widely used for:

- Penetration testing
- Network scanning
- Malware analysis

- Writing exploits

Popular tools written in Python:

- Scapy
 - SQLMap
 - Wfuzz
-

9. Cloud Computing & DevOps

Cloud platforms like AWS, Azure, and Google Cloud support Python for automation and deployment.

Python is used in:

- Infrastructure management
 - Cloud resource automation
 - Serverless applications
 - CI/CD pipelines
-

10. Scientific Computing & Research

Scientists prefer Python due to its mathematical and scientific libraries.

Libraries:

- SciPy
- SymPy
- Jupyter Notebook

Uses:

- Simulations
 - Mathematical modeling
 - Research analysis
-

11. Mobile App Development

Though less common, Python can create mobile apps.

Frameworks:

- Kivy
 - BeeWare
-

12. Education & Academics

Python is the first choice language for teaching programming in schools and universities because:

- It is easy to learn
 - Clean syntax
 - Great documentation
-

Conclusion

Python is a versatile, powerful, and beginner-friendly programming language used across nearly every field of modern technology. Whether it is IoT, AI, machine learning, data science, automation, or web development, Python provides the tools and libraries needed to build robust applications. Its simplicity and global community support make it one of the most essential programming languages in today's digital world.

5.3 Fundamentals of Python Programming Language – Long Answer

Python fundamentals include the core concepts required to write effective and efficient programs. These fundamentals form the base for IoT projects, automation, data science, and all advanced Python applications. The major fundamentals are: understanding Python as an interpreted language, working with variables and data types, importing libraries, and using flow control statements.

5.3.1 Understanding Python & Interpreted Languages

What is Python?

Python is a **high-level, general-purpose, interpreted, object-oriented programming language** designed to be easy to read and simple to write. It emphasizes code readability using indentation and supports multiple programming paradigms.

Features of Python

- Simple and easy to learn
- Interpreted
- Object-oriented
- Extensive library support
- Cross-platform
- Free & open source
- Used in IoT, AI, ML, automation, and web development

Python as an Interpreted Language

Programming languages are divided into:

- **Compiled Languages** – C, C++, Java
- **Interpreted Languages** – Python, JavaScript, Ruby

Differences between Compiler and Interpreter

Compiler	Interpreter
Converts entire code at once	Converts code line-by-line
Fast execution	Slower execution
Errors shown at the end	Errors shown immediately
Example: C, C++	Example: Python, Ruby

Why Python is Interpreted?

- Python code is executed line-by-line
- Interpreter translates source code to byte code
- Python Virtual Machine (PVM) executes bytecode

Advantages of interpreted nature

- Easy debugging
 - Suitable for beginners
 - Platform-independent
-

5.3.2 Variables, Keywords, Operators, Operands & Data Types

1. Variables in Python

A variable is a name given to a memory location where a value is stored.

Rules for Python variables

- Must start with alphabet or underscore
- Cannot start with a number
- Case-sensitive
- Can contain letters, digits, underscore

Examples

```
name = "John"  
age = 25  
pi = 3.14
```

2. Keywords in Python

Keywords are reserved words that cannot be used as variable names.

Common Python keywords

```
if, else, elif, for, while,  
break, continue,  
def, class, return,  
import, from,  
True, False, None
```

You can list keywords using:

```
import keyword  
print(keyword.kwlist)
```

3. Operators in Python

Operators perform operations on variables and values.

Types of Operators

(a) Arithmetic Operators

+ - * / % // **

Example:

```
x = 10  
y = 3  
print(x + y)    # 13
```

(b) Relational (Comparison) Operators

`== != < > <= >=`

(c) Logical Operators

`and or not`

(d) Assignment Operators

`= += -= *= /= etc.`

(e) Bitwise Operators

`& | ^ ~ << >>`

(f) Membership Operators

`in, not in`

(g) Identity Operators

`is, is not`

4. Operands

Operands are values or variables on which operators act.

Example:

```
a = 10
```

```
b = 5
c = a + b # a and b are operands, + is operator
```

5. Data Types in Python

Python is dynamically typed, meaning the type is assigned at runtime.

Basic Data Types

Type	Example
int	10
float	3.14
str	"Hello"
bool	True/False
complex	2 + 3j

Collection Data Types

Type	Description
list	Ordered, mutable
tuple	Ordered, immutable
set	Unordered, unique values
dict	Key-value pairs

Example:

```
list1 = [10, 20, 30]
dict1 = {"name": "Alex", "age": 22}
```

5.3.3 Importing Libraries

Python libraries are pre-written code that can be imported into programs to add functionality.

Types of Libraries

1. **Built-in libraries**
 - o math, os, random, datetime
2. **External libraries (install using pip)**

- numpy, pandas, requests, gpiozero (IoT)

Import Methods

1. Import whole library

```
import math
print(math.sqrt(25))
```

2. Import specific function

```
from math import sqrt
print(sqrt(25))
```

3. Import with alias

```
import numpy as np
```

Libraries make Python powerful for:

- IoT
- AI/ML
- Data processing
- Networking
- Automation

5.3.4 Flow Control, Conditional Statements & Loops

Flow control decides the execution order of statements.

(A) Conditional Statements

1. if statement

```
if condition:
    statements
```

2. if-else

```
if condition:
    print("True")
else:
    print("False")
```


3. if-elif-else

```
if a > b:
    print("a>b")
elif a == b:
    print("a=b")
else:
    print("b>a")
```

(B) Loops

Loops repeat a block of code multiple times.

1. for loop

Used when the number of iterations is known.

```
for i in range(1, 6):
    print(i)
```

2. while loop

Used when the number of iterations is unknown.

```
count = 1
while count <= 5:
    print(count)
    count += 1
```

(C) Loop Control Statements

1. break

Stops the loop immediately.

2. continue

Skips the current iteration.

3. pass

Placeholder statement; does nothing.

Example:

```
for i in range(5):  
    if i == 2:  
        continue  
    print(i)
```

Conclusion

The fundamentals of Python include understanding variables, data types, operators, importing libraries, and using flow control statements. These concepts form the base for all advanced applications of Python in IoT, machine learning, AI, automation, and Raspberry Pi programming. Knowing these fundamentals is essential for writing efficient and robust Python programs.

5.4 Sensors Interfacing

Sensor interfacing refers to the process of connecting physical sensors with a microcontroller (Arduino, Raspberry Pi, ESP32, etc.) so that the system can collect real-world data such as temperature, humidity, motion, and distance. Sensors convert physical quantities into electrical signals which can be read by a program (Python, C, Arduino IDE).

This section includes three commonly used IoT sensors:

1. Temperature & Humidity Sensor – **DHT11**
 2. Motion Sensor – **PIR Sensor**
 3. Obstacle Detection – **Ultrasonic Sensor (HC-SR04)**
-

5.4.1 Temperature and Humidity Sensor (DHT11)

The **DHT11** is a low-cost digital sensor used for measuring **temperature** and **humidity**.

Features of DHT11

- Measures temperature (0°C to 50°C)
 - Measures humidity (20%–90%)
 - Digital output
 - Single-wire communication
 - Low cost and easy to interface
-

Pin Configuration

Pin	Description
1	VCC (3.3V/5V)
2	Data
3	NC
4	GND

Working Principle

Humidity Measurement

- Uses a **moisture-sensitive resistive element**
- Resistance changes based on moisture in air
- Internal ADC converts it to digital humidity value

Temperature Measurement

- Uses **NTC thermistor**
- Resistance varies with temperature
- Sensor converts it into digital temperature output

Communication

- Uses **single-wire digital protocol**
- Microcontroller requests → Sensor responds with 40-bit data (Humidity int, Humidity decimal, Temp int, Temp decimal, Checksum)

Python Code to Read DHT11 (Raspberry Pi)

```
import Adafruit_DHT
```

```
sensor = Adafruit_DHT.DHT11
```

```
pin = 4
```

```
humidity, temperature = Adafruit_DHT.read(sensor, pin)
```

```
print("Temp:", temperature, "°C")
```

```
print("Humidity:", humidity, "%")
```

Applications of DHT11

- Weather monitoring system
- Smart home

- Greenhouse automation
 - Industrial temperature logging
 - IoT health monitoring
-

5.4.2 Motion Sensor (PIR Sensor)

The **PIR (Passive Infrared)** sensor detects motion by sensing the infrared radiation emitted by human bodies.

Features

- Detects human motion
 - Range: 5–12 meters
 - Output: HIGH (motion detected), LOW (no motion)
-

Working Principle

PIR sensor has:

- **Pyroelectric crystal** which generates an electric signal when infrared radiation changes
- **Fresnel lens** to focus infrared signals

How Motion is detected?

1. Human body emits infrared radiation
 2. When body moves → IR received changes
 3. Change is sensed → Output pin = HIGH
-

Pin Configuration

Pin	Description
-----	-------------

VCC	5V
-----	----

OUT	Digital output
-----	----------------

GND	Ground
-----	--------

Python Code (Raspberry Pi)

```
import RPi.GPIO as GPIO
```

```
import time

PIR_PIN = 17

GPIO.setmode(GPIO.BCM)
GPIO.setup(PIR_PIN, GPIO.IN)

while True:
    if GPIO.input(PIR_PIN):
        print("Motion Detected!")
    else:
        print("No Motion")
    time.sleep(1)
```

Applications of PIR

- Automatic doors
 - Motion alarm systems
 - Home security
 - Energy saving systems (lights ON/OFF)
 - Smart surveillance
-

5.4.3 Obstacle Detection using Ultrasonic Sensor (HC-SR04)

Ultrasonic sensor measures **distance** using sound waves.

Features

- Range: 2 cm to 400 cm
 - Accuracy: ± 3 mm
 - Trigger + Echo pins
 - Used in robotic cars and parking systems
-

Working Principle

Ultrasonic sensor follows **SONAR (Sound Navigation and Ranging)** principle.

Steps:

1. Trigger pin sends **8-cycle ultrasonic pulse (40 kHz)**
2. Sound wave hits object and returns

3. Echo pin becomes HIGH for a duration proportional to distance

Distance formula

$\text{Distance (cm)} = \frac{\text{Time } (\mu\text{s}) \times 0.0342}{2}$
 $\text{Distance (cm)} = \text{Time } (\mu\text{s}) \times 0.034$

Pin Configuration

Pin	Description
VCC	5V
TRIG	Trigger signal
ECHO	Echo signal
GND	Ground

Python Code (Raspberry Pi)

```
import RPi.GPIO as GPIO
import time

TRIG = 23
ECHO = 24

GPIO.setmode(GPIO.BCM)
GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)

while True:
    GPIO.output(TRIG, False)
    time.sleep(0.2)

    GPIO.output(TRIG, True)
    time.sleep(0.00001)
    GPIO.output(TRIG, False)

    while GPIO.input(ECHO) == 0:
        start = time.time()

    while GPIO.input(ECHO) == 1:
        end = time.time()

    duration = end - start
    distance = (duration * 34300) / 2

    print("Distance:", distance, "cm")
```

Applications of Ultrasonic Sensor

- Reverse parking systems
 - Robotics & obstacle avoiding robots
 - Level measurement
 - Smart waste bins
 - Smart vehicles
 - Distance monitoring in IoT
-

Conclusion

Sensor interfacing enables IoT systems to sense the physical world.

- **DHT11** measures temperature and humidity using resistive & thermistor elements
- **PIR Sensor** detects human motion using infrared radiation
- **Ultrasonic Sensor** detects obstacles using sound wave time-of-flight

These sensors are essential in smart home, automation, robotics, and IoT-based monitoring systems.

5.5 COMMUNICATING USING RASPBERRY PI – GSM INTERFACING & ON-BOARD Wi-Fi

Raspberry Pi (Rpi) supports multiple communication technologies for IoT applications, such as GSM, Wi-Fi, Bluetooth, LoRa, and Ethernet. Among these, **GSM interfacing** and **on-board Wi-Fi** are most commonly used for real-world IoT systems like home automation, surveillance, alerting, and remote monitoring.

This unit explains the **architecture, working, interfacing methods, commands, and Python programming** used for communication.

A) GSM Interfacing with Raspberry Pi

A **GSM module** (SIM800, SIM900, SIM808, A6, A7, etc.) allows Raspberry Pi to communicate using:

- **SMS (Send/Receive)**
- **Phone calls**
- **GPRS/Internet**
- **Location services (Cell tower)**

1. GSM MODULE OVERVIEW

Typical GSM modules used:

- **SIM800L**
- **SIM900A**
- **SIM808 (GPS+GSM)**
- **SIM7600 4G LTE modules**

These modules communicate with Raspberry Pi using:

UART Serial Communication

- Baud rate: **9600**
- AT Command based communication
- Voltage level required: **3.3V UART on Raspberry Pi**

2. Hardware Connections (SIM800/SIM900 → Raspberry Pi)

GSM Pin Raspberry Pi Pin

VCC	5V
GND	GND
TX	GPIO15 (RX)
RX	GPIO14 (TX)

Note: GSM modules require a minimum **2A power supply**.

3. GSM Communication Using AT Commands

AT commands (Attention Commands) are used to control GSM modules.

Important AT commands:

Purpose	Command
Check module	AT

Purpose	Command
Check SIM	AT+CPIN?
Signal strength	AT+CSQ
Network registration	AT+CREG?
Send SMS	AT+CMGS="number"
Read SMS	AT+CMGR=1
Delete SMS	AT+CMGD=1
Call	ATD<number>;
Hang up	ATH

4. Python Program to Send SMS Using GSM Module

```
import serial
import time

ser = serial.Serial("/dev/ttyS0", baudrate=9600, timeout=1)

ser.write(b'AT\r')
time.sleep(1)

ser.write(b'AT+CMGF=1\r')
time.sleep(1)

ser.write(b'AT+CMGS="+919876543210"\r')
time.sleep(1)

ser.write(b"Hello from Raspberry Pi!\r")
time.sleep(1)

ser.write(bytes([26])) # Ctrl+Z
time.sleep(1)

print("SMS Sent Successfully!")
```

5. GSM Applications in IoT

- Home security alerts
 - Fire & gas leakage alerts
 - Vehicle tracking systems
 - Remote weather stations
 - Smart agriculture (SMS alerts)
 - Industrial machine monitoring
-

B) ACCESSING ON-BOARD WI-FI ON RASPBERRY PI

Modern Raspberry Pi models (Pi 3, 4, 400, Zero W) include **built-in Wi-Fi** and can connect to 2.4GHz & 5GHz networks.

This is essential for IoT cloud communication, web servers, dashboards, and MQTT messaging.

1. Checking Wi-Fi Hardware

Run in terminal:

```
iwconfig
```

It should show:

```
wlan0    IEEE 802.11  ESSID:off/any
```

2. Connecting Wi-Fi (Using Desktop GUI)

If using Raspberry Pi OS Desktop:

1. Click **Wi-Fi icon** on top right
 2. Select your Wi-Fi network
 3. Enter password
 4. Connected!
-

3. Connecting Wi-Fi (Terminal Method)

Edit the **wpa_supplicant.conf** file:

```
sudo nano /etc/wpa_supplicant/wpa_supplicant.conf
```

Add:

```
network={
    ssid="Your_WiFi_Name"
    psk="Your_Password"
}
```

Restart Wi-Fi:

```
sudo wpa_cli -i wlan0 reconfigure
```

4. Checking Wi-Fi IP Address

```
ifconfig wlan0
```

or

```
hostname -I
```

5. Python Program to Check Wi-Fi Connectivity

```
import socket

try:
    socket.create_connection(("www.google.com", 80))
    print("Internet Connected!")
except:
    print("No Internet Connection!")
```

6. Applications of On-Board Wi-Fi

- IoT cloud communication
 - Home automation
 - Real-time dashboards
 - Remote monitoring
 - MQTT-based IoT systems
 - Web server communication (Flask/Node.js)
-

C) GSM VS Wi-Fi in IoT – Comparison Table

Feature	GSM	Wi-Fi
Range	Nationwide	Limited to router
Speed	Low	High
Cost	SIM charges	Free (home Wi-Fi)
Use-case	Remote areas	Home/Office IoT

Feature	GSM	Wi-Fi
Power	High	Low

Conclusion

GSM and Wi-Fi are essential connectivity technologies in Raspberry Pi-based IoT projects.

- **GSM** enables long-distance communication, SMS alerts, and mobile data access.
- **On-board Wi-Fi** provides fast internet access for cloud communication, dashboards, and IoT applications.

Together, they make Raspberry Pi a powerful and flexible communication hub in modern IoT systems.

5.6 CONNECTING DATABASE WITH RASPBERRY PI

Databases are essential in IoT applications for storing sensor values, device logs, user data, and monitoring history. Raspberry Pi supports several types of databases such as **SQLite**, **MySQL**, **MariaDB**, **MongoDB**, **Firebase**, and **Cloud databases**.

By connecting Raspberry Pi to a database, IoT systems can store and retrieve real-time data such as:

- Temperature & humidity values
- Motion detection logs
- Ultrasonic distance readings
- GPS location data
- Device status and sensor alerts

This makes Raspberry Pi suitable for industrial IoT, home automation, and smart monitoring applications.

A) TYPES OF DATABASES SUPPORTED BY RASPBERRY PI

1. SQLite

- Lightweight, file-based database

- No server installation required
- Best for small IoT projects
- Database stored in a single file: `.db`

2. MySQL / MariaDB

- Powerful server-based relational database
- Suitable for large IoT systems
- Supports multiple users and complex queries

3. MongoDB

- NoSQL, document-based
- Stores JSON-like data
- Good for sensor logs & big data

4. Cloud Databases

- Firebase
 - AWS DynamoDB
 - Google Cloud SQL
 - Thingspeak
 - MQTT cloud brokers (HiveMQ, Adafruit IO)
-

B) CONNECTING SQLITE DATABASE WITH RASPBERRY PI

SQLite is the simplest and most common database used with Raspberry Pi for IoT projects.

1. Installing SQLite

Run command:

```
sudo apt install sqlite3
```

Python library:

```
pip3 install sqlite3
```

(Usually preinstalled)

2. Creating Database & Table

Example database for storing sensor values:

```
import sqlite3

db = sqlite3.connect('iot_data.db')
cursor = db.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS sensor_data(
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    temperature REAL,
    humidity REAL,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
)
""")

db.commit()
db.close()
```

3. Inserting Data into SQLite

Example inserting DHT11 sensor values:

```
import sqlite3

temp = 29.4
hum = 61.0

db = sqlite3.connect('iot_data.db')
cursor = db.cursor()

cursor.execute("INSERT INTO sensor_data(temperature, humidity) VALUES (?,
?),
                (temp, hum)")

db.commit()
db.close()

print("Data Inserted Successfully!")
```

4. Fetching Data from SQLite

```
db = sqlite3.connect('iot_data.db')
cursor = db.cursor()

cursor.execute("SELECT * FROM sensor_data")
rows = cursor.fetchall()

for row in rows:
    print(row)

db.close()
```

C) CONNECTING MYSQL DATABASE WITH RASPBERRY PI

1. Install MySQL server

```
sudo apt install mariadb-server
```

2. Install Python MySQL library

```
pip3 install mysql-connector-python
```

3. Python Code to Connect MySQL

```
import mysql.connector

db = mysql.connector.connect(
    host="localhost",
    user="root",
    password="yourpassword",
    database="iot_db"
)

cursor = db.cursor()
print("Connected Successfully!")
```

4. Insert Sensor Data

```
temp = 30.1
hum = 55.2

cursor.execute("INSERT INTO sensor_values (temperature, humidity) VALUES (%s, %s)",
               (temp, hum))

db.commit()
```

D) CONNECTING RASPBERRY PI TO CLOUD DATABASE (Firebase)

1. Install Firebase library

```
pip3 install firebase_admin
```

2. Python Code

```
import firebase_admin
```

```
from firebase_admin import credentials, db

cred = credentials.Certificate("firebase_key.json")
firebase_admin.initialize_app(cred, {
    'databaseURL': 'https://your-project.firebaseio.com'
})

ref = db.reference("iot/sensor")
ref.push({
    "temperature": 31,
    "humidity": 60
})
```

E) WHY DATABASE IS IMPORTANT IN IOT USING RASPBERRY PI?

Feature	Benefits
Storage	Store continuous sensor readings
Analysis	Data analytics & graphs
History	Past data trend monitoring
Automation	Trigger actions when values exceed threshold
Cloud sync	Real-time monitoring anywhere
Scalability	Suitable for large projects

F) REAL-WORLD APPLICATIONS USING RPI + DATABASE

- Smart home temperature logging
 - Weather station data storage
 - Industrial machine monitoring
 - Smart agriculture soil/moisture logging
 - Attendance and RFID systems
 - Vehicle tracking and alert system
 - Home security system logs
-

Conclusion

Connecting a database with Raspberry Pi is a crucial component of IoT systems. Using SQLite for small projects, MySQL for large-scale systems, or cloud databases for real-time monitoring, Raspberry Pi becomes a complete data-driven IoT platform. With Python support, storing, retrieving, and analyzing sensor data becomes easy and highly efficient.

